SOP1 L1 – System plików: notatka

Oct 17, 2026 · @Andrzej

Zasady labowe i środowisko

Na labie liczy się kod na serwerze, który się kompiluje, jest sformatowany i sprawdza każdą funkcję systemową. Notatka obejmuje tutorial L1 (System plików 1); każda funkcja ma opis: do czego służy, co przyjmuje, co zwraca, przykład i pułapki.

Kompilacja

```
gcc -Wall -Wextra -Wshadow -Werror -fsanitize=address,undefined -g -o prog
prog.c
./prog
```

Flaga	Po co
`-Wall -Wextra -Wshadow`	ostrzeżenia o podejrzanym kodzie
`-Werror`	każde ostrzeżenie = błąd kompilacji (jest w przykładowym Makefile przedmiotu)
`-fsanitize=address,undefined`	wykrywa błędy pamięci, wycieki i zachowania niezdefiniowane w trakcie działania
`-g`	numery linii w raportach sanitizera
`-o prog`	nazwa programu wynikowego – po `-o` nigdy plik `.c`!

Makro ERR – obsługa błędów

```
#define ERR(source) (perror(source), fprintf(stderr, "%s:%d\n", __FILE__,
__LINE__), exit(EXIT_FAILURE))
```

Wypisuje komunikat błędu systemowego (z `errno`), plik i linię, potem kończy program. Wzorzec użycia: `if (funkcja(...)) ERR("funkcja");`

Dokumentacja

- Korzystaj z `man 3p <funkcja>` (standard POSIX), nie z `man 3` (implementacja Linuksa). Tam, gdzie Linux odbiega od standardu, trzymamy się standardu.

- `man sys_stat.h`, `man 0p dirent.h` – opisy struktur i makr z nagłówków.
- Nawigacja w `man`: `/tekst` szukaj, `n` następne, `q` wyjście, `man -k słowo` szuka stron.

Git i serwer

- Każdy etap = `git add`, `git commit -m "Etap 2 – ..."`, `git push`. Kod, którego nie ma na serwerze, nie jest oceniany.
- Serwer odrzuci push, gdy: zmieniono pliki inne niż wskazane `.c` (np. Makefile), kod nie jest sformatowany `clang-format` (plik `.clang-format` w repozytorium), kod nie kompiluje się Makefile'em z repozytorium, plik ma ponad 600 linii.
- Push raz na minutę; możliwość pushowania kończy się dokładnie z czasem zadania.

Edytor (VS Code)

- `Ctrl+Shift+P` → `C/C++: Edit Configurations (UI)` → C standard = `gnu17` (inaczej fałszywe błędy typu „mode_t is undefined").
- Format on Save + `C_Cpp: Formatting` = `clangformat`.
- Narzędzia AI (np. Copilot) są na labie zakazane: `Ctrl+,` → `disable ai features`.
- Laboratorium trwa 2 h 15 min (wtorki 14:15–16:30).

Sygnatury i obsługa błędów

Deklaracja z `man` mówi, co podać i co dostaniesz; w swoim kodzie funkcję tylko wywołujesz, a deklaracja jest już w dołączonym nagłówku.

```
DIR *opendir(const char *dirname);     // deklaracja – jest w <dirent.h>
DIR *dirp = opendir(".");              // wywołanie – to piszesz Ty
```

Algorytm: od strony man do kodu

1. SYNOPSIS – jaki `#include` i jaka sygnatura.
2. Parametr zwykły → podajesz wartość. Parametr wskaźnikowy `T *` → podajesz adres `&zmienna` albo tablicę/napis.
3. Typ zwracany → w jakiej zmiennej zapisać wynik.
4. RETURN VALUE → jak wygląda błąd (`NULL`, `-1`, wartość niezerowa, `EOF`).
5. ERRORS → możliwe kody `errno`.
6. Piszesz `if (wywołanie_z_błędem) ERR("nazwa");`

Wzorzec „Ty dajesz miejsce, funkcja je wypełnia”

```
struct stat st;              // zwykła zmienna
lstat("a.txt", &st);         // przekazujesz ADRES

char path[PATH_MAX];         // tablica
getcwd(path, sizeof(path));  // nazwa tablicy sama jest adresem – bez &
```

Masz gwiazdkę w deklaracji → pola czytasz strzałką (`dp->d_name`). Nie masz → kropką (`st.st_size`).

errno

Co to: globalna zmienna `int` z `<errno.h>`, do której funkcja systemowa zapisuje kod błędu. Wartość zwracana mówi *że* był błąd, `errno` mówi *jaki*.

Trzy zasady:

1. Sprawdzasz `errno` tylko wtedy, gdy funkcja zgłosiła błąd – przy sukcesie mogą tam być śmieci.
2. Funkcje systemowe nie zerują `errno` przy sukcesie – jeśli musisz odróżnić błąd od „niczego” (`readdir`), zerujesz sam tuż przed wywołaniem.
3. Każda kolejna funkcja (też `printf`) może `errno` nadpisać – sprawdzaj od razu.

Kod	Znaczenie	Typowa sytuacja
`ENOENT`	brak pliku/katalogu	zła ścieżka, `unlink` nieistniejącego pliku
`EACCES`	brak uprawnień	katalog bez `r` (opendir) lub bez `x` (chdir)
`ENOTDIR`	to nie katalog	`opendir` / `chdir` na zwykłym pliku
`EEXIST`	już istnieje	`mkdir` na istniejącej nazwie
`EINVAL`	zły argument	`readlink` na zwykłym pliku
`ERANGE`	wynik się nie mieści	za mały bufor w `getcwd`

Wypisanie: `perror("opendir")` → `opendir: No such file or directory` . Gdy `ERR` wywołasz bez błędu systemowego, `errno == 0` i zobaczysz absurdalne `...: Success` .

Błąd, który nie jest błędem – reaguj na konkretny kod:

```
if (unlink(name) && errno != ENOENT)
    ERR("unlink");
```

Błąd, który nie musi kończyć programu (np. zły katalog od użytkownika):

```
if (chdir(argv[i]))
{
    perror(argv[i]);
    continue;
}
```

Przeglądanie katalogu – `<dirent.h>`

Katalog to lista par „nazwa → numer i-węzła”. `readdir` daje tylko nazwę i numer; typ, rozmiar i uprawnienia bierzesz osobno przez `lstat`.

Schemat: `opendir` → `DIR *` → `readdir` w pętli → `closedir`.

Funkcja	Do czego	Przyjmuje	Zwraca	Błąd
`DIR *opendir(const char *dirname)`	otwiera katalog do czytania	ścieżkę (`"."`, `argv[1]`)	uchwyt `DIR *`	`NULL` + errno
`struct dirent *readdir(DIR *dirp)`	daje następny wpis	uchwyt z `opendir`	wskaźnik na wpis	`NULL` = koniec **albo** błąd
`int closedir(DIR *dirp)`	zamyka katalog, zwalnia pamięć	ten sam uchwyt	`0`	niezerowe

struct dirent

Według POSIX tylko dwa pola:

```
ino_t d_ino;     // numer i-węzła  → printf("%lu", (unsigned long)dp->d_ino)
char  d_name[];  // nazwa (sama nazwa, bez ścieżki!)
```

Linux ma jeszcze `d_off`, `d_reclen`, `d_type` – poza standardem, na labie ich nie używamy.

Wzór pętli (na pamięć)

```
DIR *dirp = opendir(".");
if (dirp == NULL)
    ERR("opendir");

struct dirent *dp;
while (1)
{
    errno = 0;                          // 1. zawsze tuż przed readdir
    if ((dp = readdir(dirp)) == NULL)  // 2. przypisanie w nawiasach
        break;                          // 3. koniec albo błąd
    if (strcmp(dp->d_name, ".") == 0 || strcmp(dp->d_name, "..") == 0)
        continue;                       // 4. opcjonalnie pomiń . i ..
    printf("%s\n", dp->d_name);         // 5. praca z wpisem
}
if (errno != 0)                         // 6. OD RAZU po pętli
    ERR("readdir");
if (closedir(dirp))
    ERR("closedir");
```

Wiedza z tutorialu

- Każdy katalog ma wpisy `.` (on sam) i `..` (rodzic) – to dowiązania twarde. Program liczący katalogi bez pomijania ich pokaże 2 tam, gdzie nie ma podkatalogów.
- `opendir(".")` – nie trzeba znać katalogu roboczego, kod jest prostszy.
- Kolejność wpisów z `readdir` jest dowolna – nie zakładaj sortowania ani tego, że `.` będzie pierwsze.
- Struktura, na którą wskazuje `dp`, należy do systemu: nie `free`, a nazwę potrzebną na później skopiuj.
- `ls` ukrywa nazwy od kropki; Twój program zachowuje się jak `ls -a`.
- Napisów nie porównuje się przez `==` (porównałoby adresy) – tylko `strcmp(a, b) == 0`. `strcmp` zwraca 0 dla równych napisów.
- Nie pomijaj wszystkiego, co zaczyna się od kropki – zgubisz `.gitignore` i inne ukryte pliki.
- „Bez podkatalogów” w treści zadania = nie wchodź do środka (bez rekurencji); same podkatalogi liczysz.

stat i lstat – `<sys/stat.h>`

`stat` i `lstat` wypełniają Twoją `struct stat` informacjami o pliku. Różnią się tylko przy symlinkach: `stat` idzie za linkiem i opisuje plik docelowy, `lstat` opisuje sam link.

Funkcja	Przyjmuje	Zwraca
`int stat(const char *path, struct stat *buf)`	ścieżkę + **adres** Twojej struktury	`0` sukces, `-1` błąd + errno
`int lstat(const char *path, struct stat *buf)`	jak wyżej	jak wyżej

Strona man: `man 3p fstatat` (stat i lstat są opisane razem z fstatat), pola struktury: `man sys_stat.h`.

Wywołanie krok po kroku

```
struct stat st;               // 1. miejsce na wynik (NIE wskaźnik)
if (lstat("a.txt", &st))      // 2. adres struktury, 3. sprawdzenie błędu
    ERR("lstat");
printf("%lld\n", (long long)st.st_size);   // 4. pola kropką
```

Pola, których używasz na L1

Pole	Typ	Znaczenie	Jak wypisać
`st_mode`	`mode_t`	typ pliku + uprawnienia	makra `S_IS...`; `%o` z `(unsigned) (st.st_mode & 0777)`
`st_size`	`off_t`	rozmiar w bajtach	`%lld` z `(long long)`
`st_ino`	`ino_t`	numer i-węzła	`%lu` z `(unsigned long)`
`st_nlink`	`nlink_t`	liczba dowiązań twardych	`%lu` z `(unsigned long)`
`st_mtim`	`struct timespec`	czas modyfikacji (skrót `st_mtime`)	`ctime(&st.st_mtime)`

Inne pola: `st_uid`, `st_gid` (właściciel, grupa), `st_atim` (dostęp), `st_ctim` (zmiana metadanych – to *nie* czas utworzenia), `st_blocks` (zajęte bloki – mniejsze od rozmiaru przy dziurach w pliku), `st_dev`, `st_rdev`, `st_blksize`.

Typy POSIX mają różne rozmiary na różnych systemach, dlatego w `printf` zawsze rzutujesz na znany typ. Zasada nazw: pola `st_...`, typy `..._t`.

st_mode: typ + uprawnienia w jednej liczbie

```
st_mode ósemkowo:  100644 (plik)   40755 (katalog)   120777 (link)
                   typ  prawa       typ prawa         typ  prawa
```

To dokładnie pierwsza kolumna `ls -l`: litera typu + 9 znaków `rwx`.

Makro	Prawda, gdy	Litera w `ls -l`
`S_ISREG(m)`	zwykły plik	`-`
`S_ISDIR(m)`	katalog	`d`
`S_ISLNK(m)`	link symboliczny (tylko z `lstat`!)	`l`
`S_ISFIFO(m)`	FIFO	`p`
`S_ISCHR(m)`, `S_ISBLK(m)`	urządzenie	`c`, `b`
`S_ISSOCK(m)`	gniazdo	`s`

- Makra przyjmują `st.st_mode`, nie całą strukturę. Tutorial pisze `buf->st_mode`, bo `buf` to wskaźnik z deklaracji – u Ciebie `st.st_mode`.
- Makra zwracają wartość niezerową dla prawdy i 0 dla fałszu – pisz `if (S_ISDIR(m))`, nigdy `== 1`.
- Makra `S_TYPEISMQ`, `S_TYPEISSEM`, `S_TYPEISSHM` przyjmują wskaźnik na strukturę (`&st`) – temat na SOP2.
- Plik wykonywalny to nadal zwykły plik (`-`): prawo `x` siedzi w uprawnieniach, nie w typie.

Bity uprawnień

Wagi w każdej trójce: `r = 4`, `w = 2`, `x = 1`. Trójki: właściciel, grupa, inni.

Zapis	Bity	Ósemkowo
`rw-r--r--`	110 100 100	644
`rwxr-x---`	111 101 000	750
`-w--w-r--`	010 010 100	224

Stałe pojedynczych bitów: `S_IRUSR S_IWUSR S_IXUSR` (właściciel), `S_IRGRP S_IWGRP S_IXGRP` (grupa), `S_IROTH S_IWOTH S_IXOTH` (inni).

```
if (st.st_mode & S_IWOTH)
    printf("Każdy może zapisywać!\n");
```

W terminalu: `stat -c '%n %F %a %s %h' *` – nazwa, typ, prawa, rozmiar, dowiązania. `stat -L` działa jak `stat()`, bez `-L` jak `lstat()`.

Katalog roboczy – `<unistd.h>`

Katalog roboczy (CWD) to właściwość procesu: względem niego system rozwiązuje ścieżki względne (`"."`, `"a.txt"`, nazwy z `readdir`). Każdy proces ma własny; zmiana w programie nie wpływa na powłokę, więc program nie musi wracać do startowego katalogu przed końcem.

Funkcja	Do czego	Przyjmuje	Zwraca	Terminal
`char *getcwd(char *buf, size_t size)`	odczytuje CWD (ścieżka bezwzględna)	**Twój bufor** + jego rozmiar	`buf` przy sukcesie, `NULL` przy błędzie	`pwd`
`int chdir(const char *path)`	zmienia CWD	ścieżkę względną lub bezwzględną	`0` / `-1` + errno	`cd`

```
#include <limits.h>                    // PATH_MAX
char start[PATH_MAX];
if (getcwd(start, sizeof(start)) == NULL)
    ERR("getcwd");                     // ERANGE = za mały bufor

if (chdir("dir1"))
    ERR("chdir");                      // ENOENT, ENOTDIR, EACCES
/* teraz opendir(".") i lstat("x.txt") działają wewnątrz dir1 */

if (chdir(start))                      // powrót do ZAPAMIĘTANEGO startu
    ERR("chdir");
```

Dwie drogi skanowania innego katalogu

	Droga A: `opendir(sciezka)`	Droga B: `chdir(sciezka)` + `opendir(".")`
`lstat(dp->d_name)`	❌ szuka w złym katalogu (ENOENT)	✅ działa
Co trzeba dodać	sklejanie ścieżki `snprintf(path, sizeof(path), "%s/%s", dir, name)`	`getcwd` przed i powrót `chdir(start)` po
Tutorial	wspomina	wybiera, żeby pokazać `chdir`

Przy wielu argumentach trzeba wracać do startu: dla `./prog dir1 dir2/dir3` bez powrotu druga ścieżka byłaby liczona od `dir1`. Powrót przez `chdir("..")` jest błędny – cofa o jeden poziom, a nie do startu (`dir2/dir3` → `dir2`, `/tmp` → `/`).

Prawa r/w/x dla katalogu

Prawo	Dla pliku	Dla katalogu
`r`	czytanie zawartości	odczyt listy nazw (`opendir`, `readdir`, `ls`)
`w`	zapis zawartości	tworzenie, usuwanie, zmiana nazw wpisów
`x`	uruchomienie	przeszukiwanie: wejście (`chdir`) i dostęp do plików po nazwie

Wynik testów z naszego programu `l1-2`:

Wywołanie	Błąd	Funkcja	Dlaczego
`nie_ma`	No such file or directory	`opendir`	brak katalogu
`zablokowany` (000)	Permission denied	`opendir`	brak `r`
`plik.txt`	Not a directory	`opendir`	to plik
`dir1` z prawami 444	Permission denied	`chdir`	`r` jest, brak `x`

Wniosek: niesprawdzone `chdir` przy katalogu `r--` zostawiłoby program w starym miejscu i licząc `lstat` dla nazw z innego katalogu, bez żadnego komunikatu.

`PATH_MAX` z `<limits.h>` jest bezpieczniejsze niż `#define MAX_PATH 101` z tutorialu – dłuższa ścieżka da `ERANGE`.

Parametry programu

Wszystkie argumenty trafiają do `main` jako napisy; parametry pozycyjne czytasz z `argv`, opcje typu `-n NAME` rozpoznaje `getopt`, a liczby zamieniasz przez `strtol`.

argc i argv

```
int main(int argc, char **argv)
```

Dla `./prog dir1 dir2/dir3`: `argc == 3`, `argv[0] == "./prog"`, `argv[1] == "dir1"`, `argv[2] == "dir2/dir3"`, `argv[3] == NULL`.

```
for (int i = 1; i < argc; i++)          // od 1 – argv[0] to nazwa programu
    printf("%s:\n", argv[i]);           // NIGDY printf(argv[i])
```

- Wszystko w `argv` to tekst: `"100"` to nie liczba.
- Spacja rozdziela argumenty; `"moj plik"` w cudzysłowie = jeden argument.
- `*.txt` rozwija powłoka, zanim program wystartuje.
- `printf(argv[i])` jest zabronione – argument `%d` zostałby potraktowany jak format.

usage

Komunikat „jak mnie uruchomić” przy złych argumentach. Wymagany w zadaniach przykładowych (np. zad. 1 etap 5).

```
void usage(const char *pname)
{
    fprintf(stderr, "USAGE: %s -n NAME -p OCTAL -s SIZE\n", pname);
    exit(EXIT_FAILURE);
}
/* wywołanie: */ usage(argv[0]);
```

- Na `stderr`, bo to komunikat, nie wynik. Test: `./prog 2>/dev/null` – komunikat znika.
- `exit(EXIT_FAILURE)` natychmiast kończy program (kod 1, sprawdzisz `echo $?`) – nic po `usage` się nie wykona.
- `argv[0]` – prawdziwa nazwa, pod jaką uruchomiono program.

getopt

```
#include <unistd.h>
int getopt(int argc, char *const argv[], const char *optstring);
extern char *optarg;   // wartość bieżącej opcji
extern int optind;     // indeks pierwszego argumentu, który nie był opcją
```

Do czego: jedno wywołanie = jedna kolejna opcja. Wołasz w pętli, aż zwróci `-1`. Sam pamięta, gdzie skończył (`optind`).

Sytuacja	Zwraca	`optarg`
znana opcja	jej literę, np. `'n'`	wartość (gdy w `optstring` po literze jest `:`)
nieznana opcja lub brak wymaganej wartości	`'?'` (i sam wypisuje komunikat)	–
koniec opcji	`-1`	–

`optstring` : litera = opcja bez wartości, litera + `:` = z wartością. `"n:p:s:"` = trzy opcje z wartością; `"vn:"` = `-v` bez i `-n` z wartością.

```
char *name = NULL;       // znaczniki „nie podano”
mode_t perms = -1;
ssize_t size = -1;
int c;
while ((c = getopt(argc, argv, "n:p:s:")) != -1)   // nawiasy wokół
przypisania!
{
    switch (c)
    {
        case 'n': name = optarg; break;
        case 'p': perms = strtol(optarg, NULL, 8); break;
        case 's': size = strtol(optarg, NULL, 10); break;
        case '?':
        default: usage(argv[0]);
    }
}
if (name == NULL || perms == (mode_t)-1 || size <= 0 || optind < argc)
    usage(argv[0]);      // brak opcji, zły rozmiar lub nadmiarowe argumenty
```

Śledzenie dla `./prog -n plik -v -s 10` i `"n:vs:"` : wywołania zwracają kolejno `'n'` (optarg `"plik"`), `'v'` , `'s'` (optarg `"10"`), `-1` . Działa też `-s 10 -n plik` , `-nplik` , `-vnplik` .

`name = optarg` nie kopiuje napisu – wskaźnik pokazuje na znaki już leżące w `argv` (żyją do końca programu). `NULL` = wskaźnik „nigdzie”, używany jako znacznik „jeszcze nie ustawiono”. Gdy pokazujesz na coś nietrwałego (np. `dp->d_name`), trzeba skopiować.

strtol

```
#include <stdlib.h>
long strtol(const char *str, char **endptr, int base);
```

Parametr	Znaczenie
`str`	tekst, np. `"640"`
`endptr`	adres Twojego `char *` – funkcja wpisze, gdzie skończyła czytać; `NULL` = nie sprawdzam
`base`	podstawa: `10`, `8` (uprawnienia), `16`, `0` = wykryj (`0x..`, `0..`)
zwraca	liczbę `long`; `0` gdy nic nie przeczytał; przy przepełnieniu `LONG_MAX` / `LONG_MIN` + `ERANGE`

`strtol("640", NULL, 8)` = 416 dziesiętnie = 0640 ósemkowo. `strtol("9", NULL, 8)` = 0 (9 nie jest cyfrą ósemkową).

Pełna kontrola poprawności:

```
char *end;
long v = strtol(optarg, &end, 10);
if (end == optarg || *end != '\0')   // nic nie przeczytano / zostały śmieci
    usage(argv[0]);
```

`atoi` nie odróżnia `"0"` od `"abc"` i zna tylko podstawę 10 – dlatego `strtol`.

Operacje na plikach – `<stdio.h>`

Plik obsługujesz jak katalog: `fopen` daje uchwyt `FILE *`, wszystkie operacje dostają ten uchwyt, `fclose` go oddaje. `FILE` to typ nieprzejrzysty – nie zaglądasz do jego pól.

Kolejność z tutorialu: `fopen` → (`fseek`) → zapis/odczyt → powtórz → `fclose`.

Funkcja	Do czego	Zwraca / błąd
`FILE *fopen(const char *pathname, const char *mode)`	otwiera/tworzy plik	`FILE *` / `NULL` + errno
`int fclose(FILE *stream)`	zamyka plik	`0` / `EOF`
`int fprintf(FILE *stream, const char *format, ...)`	zapis sformatowanego tekstu	liczba bajtów / ujemna

Funkcja	Do czego	Zwraca / błąd
`int fputc(int c, FILE *stream)`	zapis jednego bajtu	ten bajt / `EOF`
`int fputs(const char *s, FILE *stream)`	zapis napisu	nieujemna / `EOF`
`char *fgets(char *s, int n, FILE *stream)`	odczyt linii (z `\n`, max `n-1` znaków + `\0`)	`s` / `NULL` (koniec lub błąd)
`int fgetc(FILE *stream)`	odczyt bajtu	bajt / `EOF`
`size_t fread(void *ptr, size_t size, size_t nitems, FILE *stream)`	odczyt surowych bajtów	liczba **całych elementów**
`size_t fwrite(const void *ptr, size_t size, size_t nitems, FILE *stream)`	zapis surowych bajtów	liczba całych elementów
`int fscanf(FILE *stream, const char *format, ...)`	odczyt tekstu z konwersją	liczba udanych konwersji / `EOF`
`int fseek(FILE *stream, long offset, int whence)`	przesunięcie kursora	`0` / `-1` + errno
`long ftell(FILE *stream)`	aktualna pozycja kursora	pozycja / `-1`
`void rewind(FILE *stream)`	kursor na początek	–

Tryby fopen (napis w cudzysłowie!)

Tryb	Czyta	Pisze	Plik nie istnieje	Plik istnieje
`"r"`	tak	nie	błąd	od początku
`"r+"`	tak	tak	błąd	od początku, nic nie kasuje
`"w"`	nie	tak	tworzy	**kasuje zawartość**
`"w+"`	tak	tak	tworzy	**kasuje zawartość**
`"a"`	nie	tak	tworzy	zapis zawsze na końcu
`"a+"`	tak	tak	tworzy	zapis zawsze na końcu

- `b` (np. `"rb"`) w POSIX nic nie zmienia – jest tylko dostęp binarny; istnieje dla zgodności ze standardem C (Windows).

- Nowy plik dostaje uprawnienia `0666 & ~umask` – `fopen` nie ma parametru uprawnień.
- W trybie `"w"` istniejący plik zachowuje stare uprawnienia.

```
FILE *f = fopen("wynik.txt", "w");
if (f == NULL)
    ERR("fopen");
fprintf(f, "Liczba: %d\n", 42);
if (fclose(f))
    ERR("fclose");
```

stdout i stderr to też FILE *

`printf(...)` = `fprintf(stdout, ...)`. Trik na opcję `-o plik` (zadanie przykładowe 1):

```
FILE *out = stdout;
if (outname != NULL)
{
    out = fopen(outname, "w");
    if (out == NULL)
        ERR("fopen");
}
fprintf(out, "...");
if (out != stdout && fclose(out))   // stdout nie zamykamy
    ERR("fclose");
```

fread vs fscanf

	`fread`	`fscanf`
Co robi	kopiuje surowe bajty 1:1	czyta tekst i zamienia według formatu
Białe znaki	jak każde inne bajty	zwykle pomija
`'\0'` na końcu	**nie dopisuje**	`%s` dopisuje
Zastosowanie	dane binarne, rekordy stałej długości, cały plik	proste pliki tekstowe

Plik z tekstem `123`: `fread(buf, 1, 3, f)` daje znaki `'1','2','3'`; `fscanf(f, "%d", &x)` daje liczbę 123. Plik z 4 binarnymi bajtami liczby: tylko `fread(&x, sizeof(x), 1, f)`.

- `fread` zwraca liczbę całych elementów. Przy niepełnym odczycie `n` doczytujesz od `ptr + size*n` na `nitems - n` elementów. Mniej niż żądano = koniec (`feof`) albo błąd (`ferror`).
- Napis z `fread` : bufor o 1 większy i `buf[n] = '\0'` (zad. 4: tablica 65 znaków na 64-bajtowy tytuł).
- `fscanf` z `%s` zawsze z limitem: `"%19s"` dla `char[20]` . Sukces = liczba konwersji: `if (fscanf(f, "%d %d", &a, &b) != 2)` .
- Tutorial: w praktyce `fread` jest używany znacznie częściej niż `fscanf` ; do linii tekstu – `fgets` / `getline` + własne parsowanie.

fseek – kursor w pliku

Każdy `FILE *` pamięta pozycję (kursor). Odczyt i zapis zaczynają się od niej i przesuwają ją dalej; `fseek` przesuwa ręcznie. Po `fopen` kursor stoi na bajcie 0 (w trybie `a` zapis zawsze idzie na koniec, niezależnie od `fseek`).

`whence`	Liczy od	`fseek(f, 3, ...)`	`fseek(f, -2, ...)`
`SEEK_SET`	początku	bajt nr 3	błąd
`SEEK_CUR`	bieżącej pozycji	3 dalej	2 wstecz
`SEEK_END`	końca	3 za końcem	2 ostatnie bajty

Zastosowania:

```
/* 1. rozmiar pliku */
if (fseek(f, 0, SEEK_END))
    ERR("fseek");
long size = ftell(f);              // pozycja za ostatnim bajtem = rozmiar
if (size < 0)
    ERR("ftell");
rewind(f);

/* 2. rekord nr i stałej długości REC */
if (fseek(f, i * REC, SEEK_SET))
    ERR("fseek");

/* 3. nadpisanie fragmentu w miejscu – tryb "r+" */
if (fseek(f, 7, SEEK_SET))
    ERR("fseek");
fprintf(f, "psa!");                // "Ala ma kota" → "Ala ma psa!"
```

```
/* 4. dziura: zapis za końcem */
if (fseek(f, 1000, SEEK_SET))
    ERR("fseek");
fputc('X', f);                    // plik: 1000 bajtów zera + 'X' = 1001 B
```

Dziura: system automatycznie wypełnia lukę za końcem pliku bajtami zerowymi; długie ciągi zer mogą nie zajmować sektorów dysku. Sprawdzisz: `ls -l` (rozmiar logiczny) vs `du -h` (faktycznie zajęte) dla offsetu np. 10 000 000; podgląd bajtów: `xxd plik`.

unlink i umask

`unlink` usuwa plik, a `umask` decyduje, jakie uprawnienia dostaną pliki tworzone później przez proces. W prog12 używasz obu: najpierw usuwasz stary plik, potem ustawiasz maskę, potem `fopen`.

unlink – `<unistd.h>`

Sygnatura	`int unlink(const char *path)`
Do czego	usuwa nazwę pliku z systemu plików
Zwraca	`0` sukces, `-1` błąd + errno
Typowe błędy	`ENOENT` brak pliku, `EACCES` brak prawa `w` do katalogu, `EISDIR` / `EPERM` gdy to katalog

```
if (unlink(name) && errno != ENOENT)   // brak pliku to nie błąd
    ERR("unlink");
```

- Plik otwarty przez jakiś proces po `unlink` znika z katalogu, ale dane istnieją, dopóki ostatni proces go nie zamknie.
- Po co `unlink`, skoro `"w"` czyści plik? Bo istniejący plik zachowałby **stare uprawnienia** – `umask` działa tylko przy tworzeniu.

umask – `<sys/stat.h>`

Sygnatura	`mode_t umask(mode_t cmask)`
Do czego	ustawia maskę tworzenia plików procesu
Zwraca	**starą maskę** – nie zgłasza błędów, więc się jej nie sprawdza

Wzór, który rządzi wszystkim:

```
uprawnienia nowego pliku = to, co proponuje funkcja  &  ~umask
```

- `fopen` zawsze proponuje `0666` (`rw-rw-rw-`).
- `mkdir(path, mode)` proponuje Twoje `mode`, np. `0777`.
- Maska działa jak **sito**: zatrzymuje wybrane bity, niczego nie dodaje.

W prog12: `umask(~perms & 0777);` – maska zatrzymuje wszystko, czego nie ma w `perms`.

Krok po kroku na bitach

Krok 1 – uprawnienia to 9 bitów, każda trójka to cyfra ósemkowa (r=4, w=2, x=1).

Krok 2 – `~` odwraca każdy bit. `~750`: 111 101 000 → 000 010 111 = `027`. Na 32-bitowym `mode_t` z przodu pojawiają się dodatkowe jedynki.

Krok 3 – `&` przepuszcza bit tylko tam, gdzie obie liczby mają 1. `& 0777` usuwa przednie jedynki i zostawia 9 bitów uprawnień.

a	b	a & b
0	0	0
0	1	0
1	0	0
1	1	1

Krok 4 – złożenie. Przykład `-p 640`:

```
perms            = 110 100 000   (640)  rw- r-- ---
maska ~perms&0777= 001 011 111   (137)  co ZATRZYMAĆ
~maska           = 110 100 000   (640)
fopen proponuje  = 110 110 110   (666)
666 & ~maska     = 110 100 000   (640)  ← dokładnie perms
```

Przykład `-p 234` (Twój plik `galazka`):

```
perms            = 010 011 100   (234)  -w- -wx r--
maska            = 101 100 011   (543)
fopen proponuje  = 110 110 110   (666)
wynik            = 010 010 100   (224)  -w- -w- r--   ← grupa straciła x
```

Inne wyniki z `fopen`: `-p 755` → `644`, `-p 777` → `666`, `-p 000` → `000`, `-p 666` → `666`.

Dlaczego nie da się nadać x przez fopen

`0666` nie ma bitów `x`, a maska potrafi tylko odejmować. Pełną kontrolę daje `chmod` po utworzeniu albo niskopoziomowe `open` (L2).

Fakty o umask

- Właściwość **procesu** (jak katalog roboczy). Obowiązuje dla wszystkich plików i katalogów tworzonych później, aż ją zmienisz.
- **Nie zmienia** istniejących plików.
- Dziedziczona przez procesy potomne; zmiana w programie nie dotyka powłoki – nie trzeba jej przywracać.
- Powłoka ma własną: `umask` w terminalu (zwykle `0022` → `touch` daje `644`, `mkdir` daje `755`).

Losowanie: srand, rand, time

`rand` generuje ciąg pseudolosowy wyliczony z ziarna; `srand(time(NULL))` ustawia ziarno na bieżący czas, więc każde uruchomienie (w innej sekundzie) daje inne liczby.

Funkcja	Nagłówek	Przyjmuje	Zwraca
`void srand(unsigned seed)`	`<stdlib.h>`	ziarno	–
`int rand(void)`	`<stdlib.h>`	–	liczbę 0 … `RAND_MAX`
`time_t time(time_t *t)`	`<time.h>`	`NULL`	sekundy od 1.01.1970

```
srand(time(NULL));              // RAZ, na początku programu
long pos = rand() % size;       // liczba z 0 … size-1
char letter = 'A' + (i % 26);  // A, B, …, Z, A, …  (tutorial: 'A' + (i %
('Z' - 'A' + 1)))
```

- `srand` wołaj **dokładnie raz** – w pętli w obrębie jednej sekundy dostawałbyś ciągle tę samą liczbę.
- `rand() % 0` to dzielenie przez zero (zachowanie niezdefiniowane) – odrzuć `size <= 0` przed losowaniem.
- Tutorial mówi o limicie `RAND_MAX` (standard gwarantuje minimum 32767): w dużych plikach losowane pozycje nie sięgają dalej niż `RAND_MAX`. Na glibc `RAND_MAX` jest dużo większy – sprawdź u siebie `printf("%d\n", RAND_MAX);`.
- Znaki to liczby: `'A'` = 65, więc `'A' + 2` = `'C'`. W `printf("%c")` lub `fputc` rzutuj wyrażenie z `ssize_t` na `int`, żeby nie dostać ostrzeżenia (przy `-Werror` = błąd).

Inne operacje na systemie plików

Katalogi, uprawnienia i dowiązania: wszystkie funkcje poza `access` i `readlink` zwracają `0` przy sukcesie i `-1` + errno przy błędzie, więc używasz ich wzorcem `if (f(...)) ERR("f");`.

Funkcja	Nagłówek	Do czego	Przyjmuje	Zwraca
`int mkdir(const char *path, mode_t mode)`	`<sys/stat.h>`	tworzy pusty katalog	ścieżkę, uprawnienia (np. `0755`)	`0` / `-1` (`EEXIST`)
`int chmod(const char *path, mode_t mode)`	`<sys/stat.h>`	zmienia uprawnienia istniejącego pliku	ścieżkę, uprawnienia	`0` / `-1`
`int access(const char *path, int amode)`	`<unistd.h>`	sprawdza istnienie i prawa	ścieżkę, flagi `F_OK R_OK W_OK X_OK` (łączone `|`)	`0` = tak, `-1` = nie
`int symlink(const char *path1, const char *path2)`	`<unistd.h>`	tworzy link symboliczny `path2 → path1`	cel, nazwę linku	`0` / `-1`

Funkcja	Nagłówek	Do czego	Przyjmuje	Zwraca
`int link(const char *path1, const char *path2)`	`<unistd.h>`	tworzy dowiązanie twarde	istniejący plik, nową nazwę	`0` / `-1`
`ssize_t readlink(const char *path, char *buf, size_t bufsize)`	`<unistd.h>`	odczytuje, na co wskazuje link	link, Twój bufor, rozmiar	liczba bajtów / `-1`

mkdir

```
if (mkdir("nowy", 0755))
    ERR("mkdir");
```

Katalog musi mieć prawo `x` – bez niego nie da się wejść do środka ani sięgnąć do plików. Wynik też przechodzi przez `umask`: `mkdir(..., 0777)` przy masce `022` daje `755`.

chmod

```
if (chmod("a.txt", 0600))
    ERR("chmod");
```

Ustawia **dokładnie** podane uprawnienia, łącznie z `x` (inaczej niż `fopen` + `umask`). Zmieniać może właściciel pliku. `chown` (zmiana właściciela) wymaga superużytkownika – tutorial go pomija.

access

```
if (access("dane.txt", R_OK | W_OK) == 0)
    printf("Mogę czytać i pisać\n");
if (access("srodowisko", F_OK))
    /* nie istnieje – np. zadanie przykładowe 3 */;
```

`amode` to **nie** to samo co `mode` z `chmod`: przyjmuje flagi, nie liczbę ósemkową.

Dowiązania

- **Twarde (`link`)** – druga nazwa tego samego i-węzła, wszystkie nazwy równorzędne; plik znika po usunięciu ostatniej. Niebezpieczne (np. nazwa w publicznym katalogu daje dostęp do pliku) – na labie ich nie używamy.

- **Symboliczne (`symlink`)** – osobny plik z własnym i-węzłem, zawierający ścieżkę. Usunięcie lub przeniesienie celu = link wiszący; usunięcie linku nie rusza celu. Rozmiar linku (`st_size` z `lstat`) = długość zapisanej ścieżki.

```
if (symlink("a.txt", "link_do_a"))     // jak ln -s a.txt link_do_a
    ERR("symlink");
```

Pułapka: ścieżka względna w linku liczy się od miejsca, w którym leży link, nie od katalogu, w którym wydałeś polecenie.

```
ln -s a/file.txt b/file.txt_bak      # ❌ link w b/ szuka b/a/file.txt
ln -s ../a/file.txt b/file.txt_bak   # ✅ z b/ trzeba wyjść w górę
```

Co się przenosi	Link względny	Link bezwzględny
tylko link	zwykle się psuje	działa
tylko cel	psuje się	psuje się
cały katalog z linkiem i celem	działa	psuje się

`symlink` nie sprawdza, czy cel istnieje – zepsuty link powstaje bez błędu.

readlink

Zwraca liczbę bajtów wpisanych do bufora i **nie dopisuje `'\0'`** – robisz to sam:

```
char target[PATH_MAX];
ssize_t len = readlink("link_do_a", target, sizeof(target) - 1);  // miejsce
na '\0'
if (len < 0)
    ERR("readlink");     // EINVAL = to nie jest link
target[len] = '\0';
printf("link_do_a -> %s\n", target);   // link_do_a -> a.txt (len == 5)
```

Wiszący link to dla `readlink` nie błąd – funkcja czyta tylko zawartość linku. W terminalu: `readlink link_do_a`.

Prawo x dla katalogu w praktyce

```
chmod 444 dir1     # r-- : ls dir1 działa; cd dir1 i cat dir1/a.txt –
Permission denied
chmod 111 dir1     # --x : ls dir1 – Permission denied; cat dir1/a.txt działa
chmod 755 dir1     # przywróć
```

Buforowanie stdout i stderr

`stdout` jest buforowany, `stderr` nie – dlatego komunikaty i debug piszesz przez `fprintf(stderr, ...)`, a na `stdout` tylko wyniki.

	`stdout`	`stderr`
Na co	wyniki programu	błędy, `usage`, debug
Do terminala	buforowany liniami – wypisuje po `\n`	bez bufora – od razu
Do pliku (`>`)	buforowany blokami – zapis, gdy zbierze się dużo danych	bez bufora
Przekierowanie	`./prog > plik`	`./prog 2> plik`, wyrzucenie: `2>/dev/null`

Eksperyment z tutorialu (prog13: `printf("%d\n", i); sleep(1);` w pętli 15 razy):

1. Do terminala – liczba co sekundę.
2. `./prog13 > out.txt`, podgląd pliku w trakcie, potem Ctrl+C – plik pusty: dane czekały w buforze i przepadły przy nietypowym zakończeniu.
3. Bez `\n` (`printf("%d", i)`) do terminala – nic nie widać do końca programu.
4. To samo przez `fprintf(stderr, ...)` – zawsze od razu.

Wnioski:

- Kończ wypisywane linie znakiem `\n`.
- Do debugowania używaj `fprintf(stderr, ...)` – przy awarii programu `printf` może się nigdy nie pojawić.
- Przykład: `grep` pisze znalezione linie na `stdout`, błędy otwarcia pliku na `stderr`. Makro `ERR` też pisze na `stderr`.
- `./d2 dir1 nie_ma > wynik.txt` – wyniki trafiają do pliku, komunikat o `nie_ma` zostaje na ekranie.

Typowe błędy

Większość poniższych błędów pojawiła się w dzisiejszych ćwiczeniach; po prawej to, co je wyłapuje.

Błąd	Skutek	Poprawnie	Kto wyłapie
`while (c = getopt(...) != -1)`	`c` = 0/1, żaden `case` nie pasuje	`while ((c = getopt(...)) != -1)`	`-Wall` (+ `-Werror`)

Błąd	Skutek	Poprawnie	Kto wyłapie
`getopt` wywołany raz przed pętlą, w warunku bez przypisania	`c` się nie zmienia, pierwsza opcja przepada	przypisanie w warunku pętli	testy
`readdir` przed pętlą i w pętli	zgubiony pierwszy wpis	tylko deklaracja przed pętlą	porównanie z `ls -a`
`dp->d_name` przed sprawdzeniem `dp == NULL`	SEGV na końcu katalogu	najpierw `if (dp == NULL) break;`	UBSan: *member access within null pointer*
`errno = 0` raz przed pętlą	fałszywy błąd lub przeoczony błąd	tuż przed każdym `readdir`	–
`printf` między pętlą a `if (errno != 0)`	`errno` nadpisane	sprawdzenie od razu po pętli	–
brak `closedir`	wyciek ~32 KB na katalog	`closedir` w tym samym obrocie co `opendir`	LeakSanitizer
`closedir` poza pętlą, gdzie `dirp` nie istnieje	błąd kompilacji (zasięg)	wewnątrz bloku z `opendir`	kompilator
liczniki zadeklarowane wewnątrz `while`	zerują się przy każdym wpisie	deklaracja przed `while`	testy
`printf` wyników wewnątrz `while`	wynik po każdym wpisie	po pętli	testy
`strcmp(a, b) || strcmp(c, d)` jako „równe”	warunek zawsze prawdziwy	`strcmp(...) == 0 || strcmp(...) == 0`	testy
`a == "tekst"`	porównanie adresów	`strcmp(a, "tekst") == 0`	`-Wall`
`lstat(dp->d_name)` po `opendir(innyKatalog)`	`ENOENT` – szuka w CWD	`chdir` albo sklejanie ścieżki `snprintf`	błąd w programie
`chdir("..")` jako powrót	złe miejsce przy `dir2/dir3`, `/tmp`	`chdir(start)` z `getcwd`	testy z zagnieżdżonymi ścieżkami
niesprawdzane `chdir`	po cichu liczy zły katalog	`if (chdir(...)) ERR("chdir");`	test z katalogiem `r--`
`lstat(path, st)` / `st->st_size` przy `struct stat st`	błąd kompilacji	`&st`, `st.st_size`	kompilator
`struct stat *st; lstat(path, st);`	zapis w losowe miejsce	zwykła zmienna + `&`	ASan
`S_ISDIR(st)` / `S_ISDIR(st.mode_t)`	błąd kompilacji	`S_ISDIR(st.st_mode)`	kompilator

Błąd	Skutek	Poprawnie	Kto wyłapie
`S_ISDIR(m) == 1`	może zawieść – gwarancja to „niezero”	`if (S_ISDIR(m))`	–
`printf("%d", st.st_size)`, `%d` dla `ssize_t`	ostrzeżenie → błąd przy `-Werror`	`%lld` + `(long long)`, `%ld` + `(long)`	`-Wall`
`%d` dla uprawnień	`416` zamiast `640`	`%o` + `(unsigned)`	oko
`sizeof(buf)` dla `char *buf = malloc(100)`	8 zamiast 100	rozmiar z `malloc` lub tablica	–
`fopen(name, 'r')`	apostrofy = znak, nie napis	`fopen(name, "r")`	kompilator
`fopen(dane, "w")` zamiast `"r"`	skasowana zawartość	właściwy tryb	–
brak sprawdzenia `NULL` po `fopen`	SEGV przy błędzie	`if (f == NULL) ERR("fopen");`	ASan
brak `unlink` przed `fopen("w")`	stary plik zachowuje stare uprawnienia	`unlink` z ignorowaniem `ENOENT`	`stat -c %a`
`umask` po `fopen`	maska nie działa	przed tworzeniem pliku	`stat -c %a`
brak sprawdzenia opcji po `getopt`	`unlink(NULL)`, `rand() % 0`	`name == NULL || size <= 0` → `usage`	UBSan
brak `case '?'` / `default`	nieznane opcje ignorowane	`usage(argv[0])`	testy
`srand` w pętli	te same liczby	raz na początku	testy
brak `'\0'` po `readlink` / `fread`	`printf("%s")` czyta za bufor	`buf[len] = '\0'`	ASan
`printf(argv[i])`	`%d` w argumencie = format	`printf("%s", argv[i])`	`-Wformat-security`
`-o prog.c` w gcc	nadpisany kod źródłowy	`-o prog`	–
kompilacja bez pełnych flag	błędy wychodzą dopiero na labie	`-Wall -Wextra -Wshadow -Werror -fsanitize=address,undefined -g`	–
wklejanie kodu bez kompilacji i testów	wolna pętla poprawek	zmień → skompiluj → uruchom → porównaj	Ty

Jak czytać raport sanitizera

1. Pierwsza linia z nazwą Twojego pliku, np. `d1.c:23:27: runtime error: member access within null pointer of type 'struct dirent'` – plik, linia, kolumna, przyczyna.
2. Rodzaj awarii: `SEGV ... zero page` = dereferencja `NULL`; `LeakSanitizer: detected memory leaks` = niezwolniony zasób.
3. Stos wywołań: szukasz pierwszej ramki `in main .../plik.c:LINIA`. Ramki `libasan.so`, `libc.so`, rejestry i `BuildId` pomijasz.

Przykład wycieku: `Direct leak of 32816 byte(s) ... #2 in main test.c:10` – linia 10 to `opendir` bez `closedir`.

Pełne programy

Cztery programy skompilowane z `-Wall -Wextra -Wshadow -Werror -fsanitize=address,undefined -g` bez ostrzeżeń i przetestowane. Najpierw napisz własną wersję, potem porównuj.

1. Licznik obiektów w katalogu roboczym (zadanie 1 tutorialu)

```
#include <dirent.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>

#define ERR(source) (perror(source), fprintf(stderr, "%s:%d\n", __FILE__,
__LINE__), exit(EXIT_FAILURE))

void scan_dir(void)
{
    DIR *dirp;
    struct dirent *dp;
    struct stat st;
    int files = 0, dirs = 0, links = 0, other = 0;

    if ((dirp = opendir(".")) == NULL)
        ERR("opendir");
    while (1)
    {
        errno = 0;
        if ((dp = readdir(dirp)) == NULL)
```

```
            break;
        if (strcmp(dp->d_name, ".") == 0 || strcmp(dp->d_name, "..") == 0)
            continue;
        if (lstat(dp->d_name, &st))
            ERR("lstat");
        if (S_ISDIR(st.st_mode))
            dirs++;
        else if (S_ISREG(st.st_mode))
            files++;
        else if (S_ISLNK(st.st_mode))
            links++;
        else
            other++;
    }
    if (errno != 0)
        ERR("readdir");
    if (closedir(dirp))
        ERR("closedir");
    printf("Files: %d, Dirs: %d, Links: %d, Other: %d\n", files, dirs, links,
other);
}

int main(void)
{
    scan_dir();
    return EXIT_SUCCESS;
}
```

Wynik w katalogu z 3 plikami, 1 podkatalogiem i 1 linkiem: `Files: 3, Dirs: 1, Links: 1, Other: 0`. Wersja z tutorialu nie pomija `.` i `..`, więc pokazuje `Dirs: 3`.

2. Licznik dla katalogów z argumentów (zadanie 2 tutorialu)

Funkcja `scan_dir` identyczna jak wyżej (plus `#include <limits.h>` i `<unistd.h>`). Zmienia się tylko `main`; błędny katalog od użytkownika nie kończy programu, ale nieudany powrót – tak.

```
int main(int argc, char **argv)
{
    if (argc < 2)
    {
        fprintf(stderr, "USAGE: %s dir...\n", argv[0]);
        return EXIT_FAILURE;
    }
    char start[PATH_MAX];
    if (getcwd(start, sizeof(start)) == NULL)
```

```
        ERR("getcwd");
    for (int i = 1; i < argc; i++)
    {
        if (chdir(argv[i]))
        {
            perror(argv[i]);   /* błąd użytkownika: komunikat i dalej */
            continue;
        }
        printf("%s:\n", argv[i]);
        scan_dir();
        if (chdir(start))      /* bez powrotu kolejne ścieżki względne byłyby
złe */
            ERR("chdir");
    }
    return EXIT_SUCCESS;
}
```

```
$ ./l1-2 dir1 nie_ma dir2/dir3
nie_ma: No such file or directory
dir1:
Files: 3, Dirs: 1, Links: 1, Other: 0
dir2/dir3:
Files: 1, Dirs: 0, Links: 0, Other: 0
```

Komunikat o `nie_ma` może się pojawić przed wynikami – `stderr` nie jest buforowany, a `stdout` do terminala tak.

3. prog12 – plik o zadanej nazwie, uprawnieniach i rozmiarze (zadanie 3 tutorialu)

Zawiera również ćwiczenie z tutorialu: rozmiar zawsze równy `SIZE`.

```
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

#define ERR(source) (perror(source), fprintf(stderr, "%s:%d\n", __FILE__,
__LINE__), exit(EXIT_FAILURE))

void usage(const char *pname)
{
    fprintf(stderr, "USAGE: %s -n NAME -p OCTAL -s SIZE\n", pname);
    exit(EXIT_FAILURE);
}
```

```
void make_file(const char *name, ssize_t size, mode_t perms, int percent)
{
    FILE *f;
    umask(~perms & 0777);
    if ((f = fopen(name, "w+")) == NULL)
        ERR("fopen");
    for (ssize_t i = 0; i < size * percent / 100; i++)
    {
        if (fseek(f, rand() % size, SEEK_SET))
            ERR("fseek");
        if (fputc('A' + (int)(i % 26), f) == EOF)
            ERR("fputc");
    }
    /* dokładny rozmiar: jeśli ostatni bajt nie istnieje, dopisz tam zero */
    if (fseek(f, size - 1, SEEK_SET))
        ERR("fseek");
    if (fgetc(f) == EOF)
    {
        if (fseek(f, size - 1, SEEK_SET))   /* po odczycie przed zapisem
zawsze fseek */
            ERR("fseek");
        if (fputc('\0', f) == EOF)
            ERR("fputc");
    }
    if (fclose(f))
        ERR("fclose");
}

int main(int argc, char **argv)
{
    char *name = NULL;
    mode_t perms = (mode_t)-1;
    ssize_t size = -1;
    int c;
    while ((c = getopt(argc, argv, "n:p:s:")) != -1)
    {
        switch (c)
        {
            case 'n':
                name = optarg;
                break;
            case 'p':
                perms = (mode_t)strtol(optarg, NULL, 8);
                break;
            case 's':
```

```
                size = strtol(optarg, NULL, 10);
                break;
            case '?':
            default:
                usage(argv[0]);
        }
    }
    if (name == NULL || perms == (mode_t)-1 || size <= 0 || optind < argc)
        usage(argv[0]);
    if (unlink(name) && errno != ENOENT)
        ERR("unlink");
    srand(time(NULL));
    make_file(name, size, perms, 10);
    return EXIT_SUCCESS;
}
```

Test: `./prog12 -n r.bin -p 640 -s 100 && stat -c '%a %s' r.bin` → `640 100` przy każdym uruchomieniu; `xxd r.bin` pokazuje bajty `00` i litery (`41` = A). Bez końcówki „dokładny rozmiar” rozmiar wynosi np. 97 – tyle, ile sięgnęło najdalsze losowanie.

Wariant bez `fseek` (zapis każdego bajtu po kolei – zawsze dokładny rozmiar, ok. 10% liter):

```
int letters = 0;
for (ssize_t i = 0; i < size; i++)
{
    char ch = '\0';
    if (rand() % 100 < percent)
        ch = 'A' + (letters++ % 26);
    if (fputc(ch, f) == EOF)
        ERR("fputc");
}
```

4. Mini `ls -l` z celem linków

```
#include <dirent.h>
#include <errno.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <unistd.h>

#define ERR(source) (perror(source), fprintf(stderr, "%s:%d\n", __FILE__,
```

```
__LINE__), exit(EXIT_FAILURE))

char type_char(mode_t m)
{
    if (S_ISDIR(m))
        return 'd';
    if (S_ISLNK(m))
        return 'l';
    if (S_ISREG(m))
        return '-';
    return '?';
}

void perms_to_string(mode_t mode, char out[10])
{
    const mode_t bits[9] = {S_IRUSR, S_IWUSR, S_IXUSR, S_IRGRP, S_IWGRP,
S_IXGRP, S_IROTH, S_IWOTH, S_IXOTH};
    const char letters[] = "rwxrwxrwx";
    for (int i = 0; i < 9; i++)
        out[i] = (mode & bits[i]) ? letters[i] : '-';
    out[9] = '\0';
}

int main(void)
{
    DIR *dirp = opendir(".");
    if (dirp == NULL)
        ERR("opendir");
    struct dirent *dp;
    struct stat st;
    char perms[10];
    while (1)
    {
        errno = 0;
        if ((dp = readdir(dirp)) == NULL)
            break;
        if (strcmp(dp->d_name, ".") == 0 || strcmp(dp->d_name, "..") == 0)
            continue;
        if (lstat(dp->d_name, &st))
            ERR("lstat");
        perms_to_string(st.st_mode, perms);
        printf("%c%s %8lld %s", type_char(st.st_mode), perms, (long
long)st.st_size, dp->d_name);
        if (S_ISLNK(st.st_mode))
        {
            char target[PATH_MAX];
```

```
            ssize_t len = readlink(dp->d_name, target, sizeof(target) - 1);
            if (len < 0)
                ERR("readlink");
            target[len] = '\0';
            printf(" -> %s", target);
        }
        printf("\n");
    }
    if (errno != 0)
        ERR("readdir");
    if (closedir(dirp))
        ERR("closedir");
    return EXIT_SUCCESS;
}
```

```
-rw-r--r--        0 a.txt
drwxr-xr-x     4096 sub
lrwxrwxrwx        5 link_do_a -> a.txt
```

`out` ma 10 bajtów: 9 znaków uprawnień + `'\0'`.

Pytania kontrolne

Zakryj kolumnę z odpowiedzią i odpowiadaj z pamięci; do pytać, na które nie odpowiesz, wróć następnego dnia.

Przeglądanie katalogu

Pytanie	Odpowiedź
Ile katalogów policzy program bez pomijania `.` / `..` w katalogu bez podkatalogów?	2 – `.` i `..` to dowiązania twarde na siebie i rodzica
Jak zrobić symlink do testów?	`ln -s prog.c link.c`
Jakie pola ma `struct dirent` w Linuksie, a jakie w POSIX?	Linux: `d_ino`, `d_off`, `d_reclen`, `d_type`, `d_name`; POSIX tylko `d_ino`, `d_name`
Linux czy standard?	standard – kod przenośny między Uniksami
Wzorzec obsługi błędów?	`if (fun()) ERR("fun");` – sprawdzamy prawie każdą funkcję systemową; większość błędów kończy program, nie wszystkie
Dlaczego `opendir(".")`?	nie trzeba znać katalogu roboczego

Pytanie	Odpowiedź
Dlaczego `errno = 0` w pętli tuż przed `readdir`?	`NULL` = koniec lub błąd; inne funkcje w pętli mogły zmienić `errno`
Czy `readdir` nie mógłby sam zerować `errno`?	nie wolno na tym polegać – funkcje nie zerują `errno` przy sukcesie
Po co nawiasy w `if ((dirp = opendir(".")) == NULL)`?	`==` wiąże mocniej niż `=` – bez nawiasów do `dirp` trafi 0/1
`stat` czy `lstat` do liczenia linków i dlaczego?	`lstat` – `stat` przechodzi przez link i widzi plik docelowy
Jaki typ w `ls -l` ma plik wykonywalny?	`-` – zwykły plik; `x` to uprawnienie, nie typ

Katalog roboczy

Pytanie	Odpowiedź
Dlaczego program zapamiętuje katalog roboczy?	przy kilku ścieżkach względnych druga byłaby liczona od pierwszej
Czy program musi wrócić do startowego katalogu przed końcem?	nie – CWD to właściwość procesu, nie dotyka powłoki
Który błąd nie musi kończyć programu?	`chdir` do złego katalogu od użytkownika – komunikat i `continue`
Dlaczego nie `printf(argv[i])`?	argument `%d` zostałby potraktowany jak format
Dlaczego `chdir` do katalogu `r--` zawodzi, a `opendir` działa?	`r` = odczyt listy nazw, `x` = wejście i dostęp do plików
Dlaczego nie `chdir("..")` jako powrót?	cofa o jeden poziom, a nie do startu

Operacje na plikach (prog12)

Pytanie	Odpowiedź
Jaką maskę daje `~perms & 0777`?	odwrotność żądanych uprawnień przycięta do 9 bitów
Jak działa `'A' + (i % 26)`?	kolejne litery A–Z w kółko

Pytanie	Odpowiedź
Dlaczego rozmiar pliku zwykle różni się od `-s`?	plik kończy się na najdalszym wylosowanym bajcie; nie zawsze trafia się ostatnia pozycja
Skąd różnica przy dużych plikach (> 64K)?	według tutorialu `rand` nie sięga dalej niż `RAND_MAX`; na glibc `RAND_MAX` jest dużo większy – sprawdź
Dlaczego ignorujemy `ENOENT` przy `unlink`?	brak pliku to tu nie błąd – inaczej można by tylko nadpisywać istniejące
Po co `unlink`, skoro `w+` czyści plik?	istniejący plik zachowuje stare uprawnienia
Dlaczego nie da się nadać `x`?	`fopen` proponuje `0666`, maska tylko odejmuje
Dlaczego nie sprawdzamy błędu `umask`?	nie zwraca błędów, tylko starą maskę
Czy trzeba przywracać `umask`?	nie – zmiana jest lokalna dla procesu
Skąd zera w pliku?	zapis za końcem wypełnia lukę zerami; długie ciągi nie zajmują sektorów
Po co typy `ssize_t`, `mode_t` zamiast `int`?	zgodność z prototypami funkcji systemowych
Ile razy wywołać `srand`?	raz w procesie, z unikalnym ziarnem (np. `time(NULL)`)
Co zmienia `b` w trybie `fopen`?	w POSIX nic – jest tylko dostęp binarny
Co się stanie z plikiem po `unlink`, gdy inny proces go używa?	znika z systemu plików, ale istnieje, dopóki ostatni proces go nie zamknie
Cechy dobrej funkcji wg tutorialu?	robi jedną rzecz, jest ogólna (np. `percent` jako parametr), dane i wyniki przez parametry/wartość zwracaną, bez zmiennych globalnych

Inne operacje i buforowanie

Pytanie	Odpowiedź
`ln -s a/file.txt b/file.txt_bak` – zadziała?	nie – ścieżka liczy się od `b/`; poprawnie `../a/file.txt`
Czym różni się `amode` w `access` od `mode` w `chmod`?	`amode` to flagi `F_OK R_OK W_OK X_OK`, `mode` to uprawnienia ósemkowo

Pytanie	Odpowiedź
Co zwraca `readlink` i czego nie robi?	liczbę bajtów ścieżki; nie dopisuje `'\0'`
Dlaczego plik z `./prog13 > out` jest pusty po Ctrl+C?	stdout do pliku jest buforowany blokami, dane nie zdążyły się zapisać
Dlaczego bez `\n` nic nie widać w terminalu?	stdout do terminala wypisuje bufor po znaku nowej linii
Gdzie wypisywać debug?	`fprintf(stderr, ...)` – bez bufora

Do samodzielnego policzenia

- `666 & ~027` = ? (odp.: `~027` = 750, wynik 640)
- `strtol("077", NULL, 8)`, `strtol("077", NULL, 10)`, `strtol("077", NULL, 0)` = ? (odp.: 63, 77, 63)
- Po `fseek(f, 0, SEEK_END)` wywołujesz `fgetc(f)` – co dostaniesz? (odp.: `EOF` – za ostatnim bajtem nic nie ma)
- Dla `./prog -v -s 5 -n a` i `"vn:s:"` – co zwracają kolejne wywołania `getopt` i ile wynosi `optind` na końcu? (odp.: `'v'`, `'s'`, `'n'`, `-1`; `optind == 6`)

Źródła

Strona przedmiotu SOP (MiNI PW):

- L1 – System plików 1 (tutorial)
- L0 – Środowisko wykonania programu POSIX
- Wykład: System plików
- Harmonogram SOP1
- Użycie GITa w czasie laboratorium
- Konfiguracja IDE
- GCC Sanitizers
- Dlaczego mam problem ze zdaniem tego przedmiotu?
- Zadania przykładowe: 1, 2, 3, 4

Standard POSIX (The Open Group):

- dirent.h, readdir, opendir, closedir
- sys/stat.h, stat/lstat
- getcwd, chdir, limits.h
- getopt, strtol

- fopen, fclose, fseek, ftell, fread, fscanf, fputc
- unlink, umask, rand/srand
- mkdir, chmod, access, symlink, link, readlink
- errno, stdin/stdout/stderr

Inne:

- Linux man-pages: inode(7), readdir(3)
- cppreference: priorytet operatorów w C, wskaźniki
- GNU C Library: Setting Permissions